参考答案（书末集中）

场景 1 答案

【进1-1】确认服务状态

① **先想什么**："服务起没起"要回答两个问题：进程活着吗？端口在听吗？

② **为什么用这个命令**：`ps aux | grep labservice` 看进程；`ss -tlnp` 看所有监听端口（-t TCP、-l 监听、-n 数字显示、-p 显示进程名）。

③ **完整步骤**：

```
$ ps aux | grep -v grep | grep labservice   # 无输出 = 进程没活
$ ss -tlnp | grep 8080                     # 无输出 = 没人监听 8080
```

④ **怎么验证做对了**：两个命令都无输出，确认"进程没起、端口没听"——故障实锤，不是"起了但连不上"。

【进1-2】定位根因行

① **先想什么**：日志在哪？哪一行是根因、哪些是干扰？——看脚本配置找日志位置，再按"级别"过滤。

② **为什么用这个命令**：日志位置在 labservice.conf 的 LOGDIR 里（或脚本中写死）。用 `tail -n 20` 看最后几行（启动失败的信息都在末尾），重点看 [ERROR] / [FATAL] 行，忽略 [INFO] 心跳行。

③ **完整步骤**：

```
$ cat labservice.conf              # 找到 LOGDIR= 那一行，记下它的值（应是你的真实路径）
$ tail -n 20 ~/lab/logs/labservice.log
... [INFO] 服务启动中，配置端口=abc
... [ERROR] 端口配置非法：abc（必须是数字）    ← 根因就是这一行
$ grep -e ERROR -e FATAL ~/lab/logs/labservice.log   # 一步过滤出根因（-e = 每个 -e 后跟一个匹配模式，可一次搜多个词）
```

④ **怎么验证做对了**：你能说出"为什么排除其他行"——[INFO] 心跳行是每 3 秒一条的噪音，[ERROR] 只在出问题时出现。处理流程四步：**确定时间窗口 → 收集日志/状态/退出码 → 结合上下文定位 → 修复并复测。**

【进1-3】修复并重启

① **先想什么**：根因是"端口配置非法"，修复 = 把配置改回合法值。

② **为什么用这个命令**：改配置用 nano 编辑 labservice.conf（nano 按键：改完 Ctrl+X → Y → 回车），把 PORT=abc 改回 PORT=8080，再重新启动。

③ **完整步骤**：

```
$ nano labservice.conf   # 改完 Ctrl+X → Y → 回车        把 PORT=abc 改回 PORT=8080，保存退出
$ ./labservice.sh start &
$ ps aux | grep -v grep | grep labservice   # 进程活了
```

④ **怎么验证做对了**：进程存在，且日志末尾出现 `[INFO] 服务启动成功`。修复不是"猜"，是"改掉日志指出的那一处"。

【进1-4】验证 + 处理记录

① **先想什么**：验证要"从外到内"——端口通不通（外部视角）、日志正不正常（内部视角）。

② **为什么用这个命令**：`curl localhost:8080` 或 `nc -zv localhost 8080` 测端口；`tail -f` 看心跳在跳。

③ **完整步骤**：

```
$ nc -zv localhost 8080              # Connection to localhost (127.0.0.1) 8080 port
[tcp/http-alt] succeeded!
$ tail -f ~/lab/logs/labservice.log  # 每 3 秒一条心跳，Ctrl+C 退出
```

④ **怎么验证做对了**：端口连通 + 心跳正常，服务恢复。处理记录示例（3 行）：

1. 现象：服务启动失败。
2. 定位：日志 [ERROR] 端口配置非法（PORT=abc）。
3. 处理：改回 PORT=8080 重启，端口 8080 连通、心跳正常。已提醒同事：改配置后保存前核对格式。

场景 2 答案

说明：本场景按整体给出四步式（4 步覆盖 4 个小题），小题编号见代码注释中的"进2-x"。

① **先想什么**：磁盘满分两层看——"全局"（哪个分区满）用 `df -h`，"局部"（谁占的）用 `du` 一层层下钻，最后用 `ls -lhS` 按大小排序找元凶。清理前必须判断文件性质：model_v1.bin 是被 v2 替代的旧模型（可删），old_service.log 是旧日志（可删），tmp_cache.bin 是缓存（可删）；但如果服务正在写某个日志，那个就不能删。**注意：这些文件是搭建手册用 sudo fallocate 创建的（root 属主、目录 root 可写），普通用户直接 rm 会 Permission denied——删除也要 sudo。**

② **为什么用这个命令**：`df -h` 看分区级占用；`du -h --max-depth=1` 只下钻一层、不淹没在细节里；`ls -lhS` 用 S 参数按大小排序，一眼看到最大的文件；`rm` 删除。不用 `find -size` 是因为目录不大，逐层看更直观、更能练"下钻"的思路。

③ **完整步骤**：

```
$ df -h /mnt/lab                                    # 进2-1：使用率 90%+
$ cd /mnt/lab
$ du -h --max-depth=1 .                             # 进2-2：models/ 最大
$ du -h --max-depth=1 ./models                      # 下钻：model_v1/v2 各 70M
$ ls -lhS ./models ./logs ./cache                   # 按大小排序确认元凶
$ sudo rm ./models/model_v1.bin                          # 进2-3：删被替代的旧模型
$ sudo rm ./logs/old_service.log ./cache/tmp_cache.bin   # 删旧日志和缓存
$ df -h /mnt/lab                                    # 进2-4：使用率显著下降
```

④ **怎么验证做对了**：清理前后各跑一次 `df -h /mnt/lab`，使用率从 90%+ 降到明显更低的水平；`du -h --max-depth=1 .` 里三个子目录的大小对应减少。**收尾（安全红线，必须做）**：先执行 `cd ~` 离开挂载点；再执行 `sudo umount /mnt/lab`；用 `mountpoint -q /mnt/lab` 检查返回非 0，或用 `findmnt /mnt/lab` 确认无结果；最后删除 `~/lab/disk.img`。若提示 target is busy，检查其他终端和进程是否仍使用挂载点。

场景 3 答案

【进3-1】主机可达性

① **先想什么**：连不上先从最底层查起——机器本身通不通。

② **为什么用这个命令**：`ping localhost` 只测回环和本机网络栈，不能证明网卡、网关或外网正常；`ping 网关IP` 才是局域网层测试，但 ICMP 可能被防火墙禁用。网关地址用 `ip route` 查。再逐步测试目标 IP、端口和应用，不能只凭一次 ping 下结论。

③ **完整步骤**：

```
$ ping -c 4 localhost      # 4 个包，全通
$ ip route                 # 找到默认网关地址
$ ping -c 4 网关IP         # 局域网通（见下方 WSL 说明）
```

WSL2 读者注意：WSL2 是 NAT 网络，ping 那个网关地址**必然 100% 丢包**（而网络其实是通的）——这是 WSL2 的正常现象，不是你操作错了；用 **`ping -c 4 8.8.8.8`** 或 **`curl`** 验证连通性即可。物理机 / 云主机 / VMware 上则是正常的"局域网通"。

④ **怎么验证做对了**：输出显示 0% packet loss（WSL2 下改用 8.8.8.8 那条）。本场景是"本地 8080 服务"，ping 通说明问题在更高层（端口/进程/配置），继续下一题。

【进3-2】端口监听检查

① **先想什么**：8080 有没有人在听？被谁听？

② **为什么用这个命令**：`ss -tlnp` 列出所有 TCP 监听端口及进程。故障 A（没起）→ 无 8080；故障 B（被占）→ 8080 有监听但进程是 http.server 不是 labservice。

③ **完整步骤**：

```
$ ss -tlnp | grep 8080
# 故障A：无输出 → 服务没起
# 故障B：LISTEN 0 5 0.0.0.0:8080 ... users:(("python3",...)) → 被 python3 占着
```

④ **怎么验证做对了**：能根据 ss 输出判断出"没起"还是"被占"，并说出占用进程是谁——这一步直接决定下一步动作（启动 vs 杀占用者）。

【进3-3】实测连通

① **先想什么**："进程在听"不等于"真的能连上"——防火墙、监听地址都可能让"听着却连不上"。

② **为什么用这个命令**：`nc -zv 主机 端口` 直接发起一次连接尝试（z=只测不传数据，v=显示过程）；`curl` 还能顺带看服务是否正常应答。

③ **完整步骤**：

```
$ nc -zv localhost 8080
# 能连：Connection to localhost (127.0.0.1) 8080 port [tcp/http-alt] succeeded!
# 不能连：Connection refused（没人听）或 timeout（有防火墙丢包）
$ curl -I localhost:8080    # 看 HTTP 响应头，确认服务正常应答
```

④ **怎么验证做对了**：能区分三种结论——refused（没进程听）、timeout（防火墙/地址错）、succeeded（连通）。故障 C（监听地址错）的表现正是"localhost 能连、真实 IP 连不上"。

【进3-4】定位 + 修复 + 复述顺序

① **先想什么**：把前三题的结论拼起来，按"主机→端口→进程→日志"给故障点定位。**小提醒：日志"毫无新增"本身也是证据**——故障 B（端口被占）时请求根本没进到 labservice，日志自然干净；别把"日志没动静"误当成"没查对"。

② **为什么用这个命令**：A 类修复=启动服务；B 类修复=找到占用者（ss 输出里有 PID）评估后 kill 掉、再启动服务；C 类修复=改回正确的监听地址。

③ **完整步骤**：

```
# 故障B 修复示例：
$ ss -tlnp | grep ':8080'          # 先看原始输出，PID 在 users:(("python3",pid=1234,...)) 里
$ PIDS=$(ss -tlnp | grep ':8080' | sed -n 's/.*pid=\([0-9][0-9]*\).*/\1/p')   # 提取全部 PID
$ N=$(echo "$PIDS" | sed '/^$/d' | wc -l)   # 统计个数（sed /^$/d：/^$/=空行，d=删除——整句=删掉空行再数）
$ if [ "$N" -ne 1 ]; then echo "检测到 $N 个进程占用 8080：$PIDS ——先列出来人工判断，停止处理"; exit 1; fi   # 不是唯一 PID 就停，绝不自动取第一个
$ OCC_PID="$PIDS"
$ ps -p "$OCC_PID" -o pid=,args= | grep -q "python3 -m http.server 8080" || { echo "PID $OCC_PID 的命令行不是本场景的 http.server，停止处理"; exit 1; }   # 核对命令行确实是本场景进程才动手
$ kill "$OCC_PID"                      # 先优雅终止
$ ./labservice.sh start &       # 重启服务（& 别漏，否则占住终端）
$ nc -zv localhost 8080         # 验证连通
```

sed 拆解（本册第一次出现，正文未教，这里一次讲清）：-n=只打印被处理的行；s/旧/新/p=替换并打印；\(...\)=圈住要保留的部分；\1=用圈住的内容替换整行——整句 = 把含 pid= 的行替换成 pid= 后面的数字。

④ **怎么验证做对了**：服务连通。四步顺序脱口而出：**先记录各层证据，再按"确定时间窗口 → 收集日志/状态/退出码 → 结合上下文定位 → 修复并复测"处理**，不能只凭单次 ping 或单条命令下结论。

场景 4 答案

【进4-1】部署 + 确认

① **先想什么**：部署 = 后台跑起来 + 退出终端也不死。

② **为什么用这个命令**：nohup 命令 & 让进程忽略挂断信号（终端关了它也不死）；确认用 ps + ss 双保险。

③ **完整步骤**：

```
$ nohup ./labservice.sh start &
$ ps aux | grep -v grep | grep labservice   # 在
$ ss -tlnp | grep 8080                    # 在听
```

④ **怎么验证做对了**：进程和端口双重确认。完成后用本场景 PID 停止服务，再检查 PID 文件、端口和进程均已消失；日志本体在 LOGDIR 里。

【进4-2】揪出死循环进程

① **先想什么**：死循环通常会持续占用 CPU；先用资源观察确认，再只操作本场景保存的 PID。

② **为什么用这个命令**：`top` 按 P 观察高 CPU 进程；`loop.pid` 是搭建时记录的本场景 PID。先发 TERM 给程序清理机会；实验脚本会用 trap 忽略 TERM，所以确认仍在后才使用 KILL。

③ **完整步骤**：

```
$ loop_pid=$(cat loop.pid)
$ ps -p "$loop_pid" -o pid=,args= # 必须确认命令行属于 loop.sh
$ kill "$loop_pid"
$ ps -p "$loop_pid" # 仍在运行才进行下一步
$ kill -9 "$loop_pid" # 最后手段
$ rm -f loop.pid loop.log
$ top # CPU 应降回正常水平
```

④ **怎么验证做对了**：确认 loop.sh 的 PID 已不再存在、CPU 占用下降、`loop.pid` 已删除。不能根据 PID 文件直接杀进程，必须先用 `ps` 核对命令归属，防止 PID 复用误杀其他进程。

【进4-3】GPU / 文件占用定位

① **先想什么**：GPU 显存被谁占着？——技能本质是"资源 → 占用它的进程"。

② **为什么用这个命令**：`nvidia-smi` 看显存使用和占用进程 PID；`fuser -v /dev/nvidia*` 反查"谁占着 GPU 设备"。无 GPU 的等价题：`fuser -v 某个文件` 反查"谁占着这个文件"——命令和思路完全一样。

③ **完整步骤**：

```
$ nvidia-smi                    # GPU 题：看 Processes 区块里的 PID
$ fuser -v /dev/nvidia*         # 哪个进程占着 GPU
$ fuser -v ~/lab/logs/labservice.log # 无 GPU 替代：哪个进程占着这个文件（选正被进程打开的文件—labservice.log 被 python3 持续打开着）
```

④ **怎么验证做对了**：fuser 输出"文件: 进程名(PID)"格式，与 ps 结果互证。fuser 只显示"打开着该文件"的进程——所以要做实验，得选一个正被进程持续打开的文件（如服务的日志），而不是写完就关的 pid 文件。核心能力不是记命令，是"从资源反查进程"这个思路——磁盘满（du 找文件）、端口被占（ss 找进程）、GPU 被占（fuser），同一种思维。

【进4-4　】开机自启

① **先想什么**：让 systemd 这个大管家替你"开机自动拉起服务"。

② **为什么用这个命令**：写一个 unit 文件描述服务，`systemctl enable` 注册开机自启，`systemctl start` 立即生效。无 systemd 的环境说明 rc.local 或 crontab @reboot 方案即可。

③ 完整步骤：

```
$ EXEC_PATH="$(readlink -f ~/lab/labservice.sh)"   # 先拿真实绝对路径（readlink -f 把 ~ 展开成完整路径）
$ echo "$EXEC_PATH"                                # 只看一眼，确认输出是完整真实路径（这一步就是用它替换下面 %s 的值）
$ printf '[Unit]\nDescription=Lab fake service\n[Service]\nExecStart=%s start\nRestart=on-failure\nRestartSec=2\n[Install]\nWantedBy=multi-user.target\n' "$EXEC_PATH" | sudo tee /etc/systemd/system/labservice.service >/dev/null   # 动态生成 unit，不手写占位路径
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now labservice
$ sudo reboot          # 重启验证
$ systemctl status labservice   # active (running) = 自启成功
$ sudo systemctl disable --now labservice && sudo rm /etc/systemd/system/labservice.service && sudo systemctl daemon-reload   # 练习完恢复原状：先 disable 停止、再删 unit、最后 daemon-reload 让 systemd 重新读配置
```

reboot 会断开当前所有连接：请在本地终端做。**注意：`systemctl restart` 不能代替重启**——它只验证"服务能跑起来"，验不到"开机自启"（那是 `enable` 的职责），要真验证得重启一次。在 WSL 里 `sudo reboot` 是空操作（不会真的重启）；请从 Windows 侧执行 `wsl --shutdown`（或 `wsl --terminate 发行版名`），再重新进入发行版看 `systemctl status labservice`。

④ 怎么验证做对了：重启后服务自动在跑。养成习惯：改完 unit 文件必须 `daemon-reload`，这是新手最容易漏的一步。

Restart=on-failure 验证（Ubuntu 26.04 + systemd 实测通过，实测现象附在各步骤后）：

```
$ sudo systemctl start labservice
$ sudo ss -tlnp | grep :8080   # 输出如 users:(("python3",pid=1234,...))—记下这个唯一 PID（unit 以 root 运行，普通用户看不到进程名，要用 sudo）
   安全警告：不要用 pkill -f 'http.server 8080'—它按整条命令行模糊匹配，会误杀别的同名 http.server。
$ PID=1234   # 换成上一步 ss 显示的唯一 PID（如 ss 显示多个 pid，先逐个 ps 核对，只处理本场景那个）
$ ps -p "$PID" -o pid=,args=   # 人工核对命令行确实是本场景子服务（含 http.server 8080），不是别的进程
$ sudo kill -9 "$PID"   # 只杀这一个 PID；父服务随后以非 0 退出，触发 Restart=on-failure
$ for i in $(seq 1 30); do systemctl is-active --quiet labservice && ss -tlnp | grep -q :8080 && break; sleep 1; done   # 轮询最多 30 秒：等 unit 恢复 active 且端口恢复（实测恢复约 3~5 秒、视机器而定；固定 sleep 2 会看到"还没恢复"的假象）
$ systemctl status labservice   # 预期：状态仍为 active (running)，Main PID 变了
$ journalctl -u labservice -n 10          # 预期：看到 systemd 的英文原话 "Scheduled restart job, restart counter is at 1."（NRestarts ≥ 1）
```

实测（Ubuntu 26.04）：kill -9 后约 3~5 秒 unit 自动重启（视机器而定）、MainPID 由旧值变为新值。**注意：PID 每次随机分配——以你机器上 ss 和 systemctl status 的实际显示为准，不要对照固定数字。**NRestarts=1；systemctl stop 后 unit inactive 且不自动重启。若你的环境现象不符 → 记录实际环境与现象，如实记为"环境限制"。**正常停止对照**：`sudo systemctl stop labservice` 后 `systemctl status labservice` 应为 inactive（dead）且 **不自动重启**（父进程 trap 收尾返回 0，不会触发 on-failure）；`sudo systemctl start labservice` 再恢复。两项都通过才算本题闭环：异常→自动重启，正常→安静退出。

场景 5 答案

【进5-1】资源巡检脚本

① **先想什么**：把三个手动命令装进一个文件，开头声明解释器。

② **为什么用这个命令**：df（磁盘）、free（内存）、uptime（负载）都是"看一眼"类命令，脚本就是它们的组合 + 中文注释。

③ **完整步骤**：

```
$ nano check_res.sh # 写完 Ctrl+X → Y → 回车保存退出
# ↓ 从 #!/bin/bash 开始到下面的分隔线为止，全部是【脚本内容】——整段粘贴进 nano
#!/bin/bash
echo "===== 磁盘 ====="; df -h /
echo "===== 内存 ====="; free -h
echo "===== 负载 ====="; uptime
```

↑以上整块就是【脚本内容】（复制到脚本文件里）；下面这条是在【终端】执行的命令，**不要**复制进脚本：

```
$ chmod +x check_res.sh && ./check_res.sh
```

④ **怎么验证做对了**：一条命令输出三段摘要，和手动敲三条完全一致。这一步完成了"手动 → 脚本"的第一次跨越。

【进5-2】服务健康检查脚本

达标版：用 pgrep 或 ps -p 判断进程在不在即可达标；下面的"四要素认领"是强化版（防 PID 复用误判），先看达标版思路，学有余力再上强化。

① **先想什么**："健康"要查三层：进程在不在、端口通不通、日志新不新。

② **为什么用这个命令**：读 `labservice.pid` + `ps` 核对命令行查进程（不用 pgrep -f，避免误匹配同名脚本）；`nc -z` 查端口；`stat -c %Y 日志文件` 拿日志最后修改时间，和当前时间比较判断"最近有没有更新"。

③ **完整步骤**：

```
$ nano health_check.sh # 写完 Ctrl+X → Y → 回车保存退出
# ↓ 从 #!/bin/bash 开始到下面的分隔线为止，全部是【脚本内容】——整段粘贴进 nano
#!/bin/bash
LOG=~/lab/logs/labservice.log
[ -f ~/lab/labservice.pid ] || { echo "进程：PID 文件不存在，服务没在运行"; exit 1; }
PID=$(sed -n '1p' ~/lab/labservice.pid)
SPATH=$(sed -n '2p' ~/lab/labservice.pid)
INSTDIR=$(sed -n '3p' ~/lab/labservice.pid)
TARGET_SH=$(readlink -f ~/lab/labservice.sh 2>/dev/null || echo '')
case "$PID" in
  ''|*[!0-9]*) echo "进程：PID 文件内容无效（$PID），请先 ./labservice.sh status 排查"; exit
1 ;;
esac
CURDIR=$(readlink -f "/proc/$PID/cwd" 2>/dev/null || echo '')
if [ -n "$SPATH" ] && [ -f "$SPATH" ] && [ "$SPATH" = "$TARGET_SH" ] && [ "$CURDIR" =
"$INSTDIR" ] && ps -p "$PID" -o args= 2>/dev/null | grep -q "labservice.sh"; then
  echo "进程：正常（PID=$PID，四要素已核对：路径=目标脚本、cwd=记录目录、命令行=本脚本）"
else
  echo "进程：实例不符或已过期——$PID 不是本场景服务（跨目录同名实例 / PID 复用 / 文件过期），
请先 ./labservice.sh status 排查"; exit 1
fi
# 本场景 PID 文件三行（父 PID / 脚本绝对路径 / 启动工作目录）：核对 PID 数字 + 记录路径=目标
```

```
脚本 + /proc/PID/cwd=记录目录 + 命令行含 labservice.sh，四要素全对齐才认（不用 pgrep -f）；只用它做判断、不做 kill
nc -z localhost 8080 >/dev/null 2>&1 && echo "端口：正常" || echo "端口：不通"
if [ -f "$LOG" ]; then
  [ $(( $(date +%s) - $(stat -c %Y "$LOG") )) -lt 300 ] && echo "日志：5分钟内更新过" || echo "日志：5分钟没动静了"
else
  echo "日志：文件不存在（先确认 LOG 路径和权限）"
fi
# $(( )) = 算术运算（两数相减）；date +%s = 当前秒数；stat -c %Y = 文件最后修改时间（秒）
```

↑以上整块就是【脚本内容】（复制到脚本文件里）；下面这条是在【终端】执行的命令，**不要**复制进脚本：

```
$ chmod +x health_check.sh && ./health_check.sh
```

④ **怎么验证做对了**：三条中文结论准确反映当前状态。故意杀掉服务再跑一次，结论应变为"挂了/不通/没动静"；把 LOG 改成不存在的路径再跑，应打印"日志：文件不存在"而不是报错——五种情况（不存在/空/坏行/正常/权限不足）都要有明确输出。

【进5-3】变量、判断与注释

① **先想什么**：脚本是写给人看的——变量消除"魔法数字"，判断增加分支，注释讲清意图。

② **为什么用这个命令**：把重复出现的路径/端口提成变量（以后改一处即可）；用 if 结构替代一长串 &&||（可读性更好）；每段逻辑前加 # 注释。**进程检查不用 pgrep -f**（按整条命令行模糊匹配会误判同名脚本）：读本场景 PID 文件（父进程 PID）+ ps 核对命令行是本脚本，缺失/过期/复用一律明确报错并返回非 0。

③ **完整步骤**：

```
#!/bin/bash
# 健康检查脚本 v2：检查 labservice 的进程、端口、日志
PORT=8080                          # 服务端口
LOG=~/lab/logs/labservice.log   # 日志路径
echo "== 检查结果 =="
PID_FILE="${PID_FILE:-$HOME/lab/labservice.pid}"    # 本场景服务的 PID 文件；${变量:-默认值}=环境里没传就取默认（测试时可用假 PID 文件覆盖，不动真实文件）
if [ ! -f "$PID_FILE" ]; then
  echo "进程：PID 文件不存在，服务没在运行"
  exit 1
fi
PID=$(sed -n '1p' "$PID_FILE")
SPATH=$(sed -n '2p' "$PID_FILE")
INSTDIR=$(sed -n '3p' "$PID_FILE")
TARGET_SH=$(readlink -f ~/lab/labservice.sh 2>/dev/null || echo '')
case "$PID" in
  ''|*[!0-9]*) echo "进程：PID 文件内容无效（$PID），请先 ./labservice.sh status 排查"; exit 1 ;;
esac
CURDIR=$(readlink -f "/proc/$PID/cwd" 2>/dev/null || echo '')
if [ -n "$SPATH" ] && [ -f "$SPATH" ] && [ "$SPATH" = "$TARGET_SH" ] && [ "$CURDIR" = "$INSTDIR" ] && ps -p "$PID" -o args= 2>/dev/null | grep -q "labservice.sh"; then
  echo "进程：正常（PID=$PID，四要素已核对：路径=目标脚本、cwd=记录目录、命令行=本脚本）"
else
  echo "进程：实例不符或已过期—$PID 不是本场景服务（跨目录同名实例 / PID 复用 / 文件过期），请先 ./labservice.sh status 排查"
  exit 1
fi
```

```
# 端口：nc 探测 8080 是否在听（连通=在听）
if nc -z localhost $PORT >/dev/null 2>&1; then
  echo "端口：正常"
else
  echo "端口：不通"
fi
# 日志：先判断文件存在，再 stat（不存在不报无关错误）
if [ -f "$LOG" ]; then
  NOW=$(date +%s)
  LAST=$(stat -c %Y "$LOG")
  if [ $(( NOW - LAST )) -lt 300 ]; then
    echo "日志：5 分钟内更新过"
  else
    echo "日志：5 分钟没动静了"
  fi
else
  echo "日志：文件不存在（检查 LOG 路径）"
fi
```

check_res.sh 完整 v2（照此模式改，题 5-1 的完整版即本模式）：

```
#!/bin/bash
# check_res.sh v2 —— 资源巡检 + 告警（变量、判断、注释版）
THRESHOLD=80   # 变式：改成 90 试试，再仿照根分区加一行 df /home 的检查
# 磁盘
USE=$(df / | tail -1 | awk '{print $5}' | tr -d '%')  # 取根分区使用率（去掉 %）
if [ "$USE" -gt "$THRESHOLD" ]; then
  echo "磁盘告警：$USE%"
else
  echo "磁盘正常：$USE%"
fi
# 内存：用 grep '^Mem:' 按行名识别（不靠行号，free 版本换了也不怕）
TOTAL=$(free -m | grep '^Mem:' | awk '{print $2}')
USED=$(free -m | grep '^Mem:' | awk '{print $3}')
if [ -z "$TOTAL" ] || [ -z "$USED" ]; then
  echo "内存：解析不到 Mem 行（free 输出结构不同），先单独跑 free -m 看实际列再改 awk"
elif [ "$TOTAL" -gt 0 ]; then
  PCT=$(( USED * 100 / TOTAL ))
  if [ "$PCT" -gt "$THRESHOLD" ]; then
    echo "内存告警：$PCT%"
  else
    echo "内存正常：$PCT%"
  fi
else
  echo "内存：总量为 0，无法计算"
fi
# 负载
uptime
```

④ **怎么验证做对了**：三种分支都要跑，**全程不碰真实的 labservice.pid**（真实服务保持运行、端口不被误停）：① 服务在跑（先 `./labservice.sh start &`）：应打印"进程：正常"且退出码 0；② 不存在的假路径：`PID_FILE="$HOME/lab/not-exist.pid" ./health_check.sh` → 打印"PID 文件不存在"且退出码非 0；③ 写假 PID：`printf '99999\n/fake/path\n/fake/dir\n' > "$HOME/lab/fake.pid"`，再 `PID_FILE="$HOME/lab/fake.pid" ./health_check.sh` → 打印"实例不符或已过期"且退出码非 0。测完 `rm -f "$HOME/lab/fake.pid"` 清理，并确认 `cat ~/lab/labservice.pid` 内容未变、`./labservice.sh status` 仍显示运行中。可读性：让没学过脚本的人读一遍能说出"它在查什么"。

【进5-4　】验证 + 定时巡检

① **先想什么**：脚本要定期自动跑，用 crontab 定计划。

② **为什么用这个命令**：`crontab -e` 编辑定时表，格式"分 时 日 月 周 命令"；`0 9 * * *` 表示每天 9:00。

③ **完整步骤**：

```
$ ./check_res.sh > ~/lab/巡检报告_$(date +%F).txt   # 先手动验证输出正确
$ CRON_HOME="$HOME"   # 用真实家目录的绝对路径（cron 的环境和终端不同，绝对路径最稳）
$ (crontab -l 2>/dev/null; printf '0 9 * * * %s/lab/check_res.sh >> %s/lab/巡检记录.log
2>&1\n' "$CRON_HOME" "$CRON_HOME") | crontab -   # 动态生成真实绝对路径，不用手写占位
$ crontab -l        # 确认已写入（能看到真实路径）
$ crontab -l | grep -v check_res.sh | crontab -   # 练习完删除这条任务，恢复原状态
```

④ **怎么验证做对了**：`crontab -l` 能看到计划；把时间改成"两分钟后"测试一次，日志文件按时新增内容，再改回 9 点。注意：cron 的环境和终端不一样（PATH 很短、展开规则也有差异）——定时任务里的路径一律写绝对路径，最稳。**WSL 用户多一句：**空闲时 WSL 会挂起、cron 服务也跟着停——测试时别关终端（或改用真机/云主机验证），免得"等半天日志没动静"。这两个脚本（check_res.sh、health_check.sh）可直接放进脚本目录。

附录 A · 练习用脚本（复制保存即可）

A1. labservice.sh（假推理服务，支持 start / stop / status）

附录脚本总警告：脚本内含自动删除命令（rm -f 清 PID 文件、stop 时结束子进程）——全部只作用于练习目录内自己的产物，不会碰其他路径；但仍需遵守：① 只在专用 Ubuntu 虚拟机或可丢弃实验环境运行；② 不把脚本复制到真实服务器使用；③ 每次实验前先 `cd ~/lab` 并确认 labservice.conf 的 LOGDIR 指向 ~/lab/logs；④ 练完用 `./labservice.sh stop` 收尾并复查现场（PID 文件已删、端口已释放）。

```
#!/bin/bash
# labservice.sh - 练习用假服务，仅在练习目录使用
# 用法：./labservice.sh {start|stop|status}
# PID 文件三行：父进程 PID / 脚本绝对路径 / 启动工作目录 —— 实例唯一标识
# stop 通知父进程，由父进程 trap 负责停子服务并 exit 0
set -u

DIR="$(cd "$(dirname "$0")" && pwd)"
CONF="$DIR/labservice.conf"
PID_FILE="$DIR/labservice.pid"

if [ ! -f "$CONF" ]; then
  echo "配置文件不存在：$CONF"
  exit 1
fi
source "$CONF"
mkdir -p "$LOGDIR" 2>/dev/null
if [ ! -d "$LOGDIR" ] || [ ! -w "$LOGDIR" ]; then
  echo "日志目录不可写：$LOGDIR"
  exit 1
fi
LOG="$LOGDIR/labservice.log"

SRV=""                          # 子服务 PID（全局，供 trap 收尾）

parent_is_ours() {              # 三要素认领：PID 文件记录的实例与本脚本一致
  [ -f "$PID_FILE" ] || return 1
  local pid spath instdir curdir args
  pid=$(sed -n '1p' "$PID_FILE" 2>/dev/null) || return 1
  spath=$(sed -n '2p' "$PID_FILE")
  instdir=$(sed -n '3p' "$PID_FILE")
  # 1) 路径一致：PID 文件是本脚本实例写的（同目录同文件），别的目录实例路径不同
  [ "$spath" = "$(readlink -f "$0")" ] || return 1
  # 2) PID 有效且命令行确实是本脚本
  args=$(ps -p "$pid" -o args= 2>/dev/null || true)
  case "$args" in
    *"labservice.sh"*) : ;;
    *) return 1 ;;
  esac
  # 3) 工作目录一致：PID 复用成别目录同名脚本时，cwd 不同即拒绝
  curdir=$(readlink -f "/proc/$pid/cwd" 2>/dev/null || echo '')
  [ "$curdir" = "$instdir" ] || return 1
  return 0
}
```

```
stop_service() {             # 只由父进程的 trap 调用：停子服务、清 PID 文件
  if [ -n "$SRV" ]; then
    kill -TERM "$SRV" 2>/dev/null
    wait "$SRV" 2>/dev/null
    kill -0 "$SRV" 2>/dev/null && kill -KILL "$SRV" 2>/dev/null
  fi
  rm -f "$PID_FILE"
  echo "[$(date '+%F %T')] [INFO] 收尾完成：子服务已停、PID 文件已清理" >> "$LOG" 2>/dev/null
|| true
}

stop_cmd() {                 # ./labservice.sh stop：通知父进程
  if [ ! -f "$PID_FILE" ]; then
    echo "本场景没有可停止的服务（PID 文件不存在）"
    exit 0
  fi
  if ! parent_is_ours; then
    echo "[拒绝] PID 文件不属于本实例（跨目录同名 / PID 复用 / 文件过期），已停止处理，不影响其
他实例"
    exit 1
  fi
  local pid
  pid=$(sed -n '1p' "$PID_FILE")
  echo "发送停止信号给父进程 PID=$pid（由它负责停子服务并清理）"
  kill -TERM "$pid" 2>/dev/null || true
  local i
  for i in $(seq 1 50); do
    if ! kill -0 "$pid" 2>/dev/null && [ ! -f "$PID_FILE" ] && ! ss -tln 2>/dev/null | grep -
q ":$PORT "; then
      echo "服务已完全停止（父进程退出、PID 文件已清理、端口已释放）"
      exit 0
    fi
    sleep 0.2
  done
  echo "[警告] 10 秒内未完全停止，请检查：ps -p $pid、ls -l $PID_FILE、ss -tlnp | grep :$PORT"
  exit 1
}

status_cmd() {
  if [ ! -f "$PID_FILE" ]; then echo "状态：未运行"; exit 1; fi
  if ! parent_is_ours; then echo "状态：PID 文件不属于本实例（跨目录同名 / 过期 / PID 复用）——
最常见是服务已停、只剩残留文件：确认端口没被占，直接 ./labservice.sh start 重开即可"; exit 1; fi
  echo "运行中：父进程 PID=$(sed -n '1p' "$PID_FILE")，端口=$PORT"
  exit 0
}

start_service() {
  if ss -tlnp 2>/dev/null | grep -q ":$PORT "; then
    echo "[ERROR] 端口 $PORT 已被占用：$(ss -tlnp 2>/dev/null | grep ":$PORT ")"
    return 1
  fi
  if parent_is_ours; then
    echo "服务已经运行：父进程 PID=$(sed -n '1p' "$PID_FILE")"
    return 1
  fi
```

```
  [ -f "$PID_FILE" ] && rm -f "$PID_FILE"   # 清掉过期残留
  echo "[$(date '+%F %T')] [INFO] 服务启动中，配置端口=$PORT" >> "$LOG"
  trap 'stop_service; exit 0' TERM
  trap 'stop_service; exit 130' INT
  if ! [[ "$PORT" =~ ^[0-9]+$ ]]; then
    echo "[$(date '+%F %T')] [ERROR] 端口配置非法: $PORT（必须是数字）" >> "$LOG"
    return 1
  fi
  python3 -m http.server "$PORT" >/dev/null 2>>"$LOG" &
  SRV=$!
  { echo "$$"; echo "$(readlink -f "$0")"; echo "$DIR"; } > "$PID_FILE"
  sleep 1
  if ! kill -0 "$SRV" 2>/dev/null; then
    echo "[$(date '+%F %T')] [ERROR] 端口 $PORT 监听失败" >> "$LOG"
    rm -f "$PID_FILE"
    return 1
  fi
  echo "[$(date '+%F %T')] [INFO] 服务启动成功，父进程 PID=$$ 子进程 PID=$SRV，端口=$PORT" >>
"$LOG"
  echo "服务已启动（父进程 PID=$$）。若没回到命令行，说明在【前台】跑——按 Ctrl+C 会停掉服务；正
确用法：./labservice.sh start &"
  while true; do
    echo "[$(date '+%F %T')] [INFO] 服务心跳正常" >> "$LOG"
    # 子服务退出检测：kill -0 对 zombie 仍为真，所以再查进程状态 Z
    if ! kill -0 "$SRV" 2>/dev/null; then
      echo "[$(date '+%F %T')] [ERROR] 子服务已退出，父服务以非 0 退出（供 Restart=on-failure
触发）" >> "$LOG"
      rm -f "$PID_FILE"
      exit 1
    fi
    STAT=$(ps -o stat= -p "$SRV" 2>/dev/null | tr -d ' ')
    case "$STAT" in
      Z*) echo "[$(date '+%F %T')] [ERROR] 子服务已成 zombie（被外部 kill -9），父服务以非 0 退
出（供 Restart=on-failure 触发）" >> "$LOG"
          rm -f "$PID_FILE"
          exit 1 ;;
    esac
    sleep 3
  done
}

case "${1:-}" in
  start) start_service ;;
  stop) stop_cmd ;;
  status) status_cmd ;;
  *) echo "用法：$0 {start|stop|status}"; exit 1 ;;
esac
```

PID 文件三行 = 实例唯一标识：父进程 PID / 脚本绝对路径 / 启动工作目录。stop/status 按"路径一致 + 命令行是本脚本 + 工作目录一致"三要素认领——两个目录各跑同名脚本也不会误认。子服务退出检测：kill -0 + 进程状态 Z（zombie）双查——被外部 kill -9 后即使成为 zombie 也能在下一轮心跳检出 → 父进程 exit 1（供 Restart=on-failure）；正常 stop/systemd TERM → trap 收尾 exit 0。请分别复核：正常 start/status/stop、Ctrl+C、重复 start、异常退出和过期 PID 文件；收尾时确认 PID 文件已删、端口不再监听、练习进程不再存在。

高级 Shell 构件速查（附录脚本用到的语法，正文未逐一展开——允许照抄运行、边用边懂；排错时翻回这张表）

构件	用途	本脚本哪里用到
`set -u`	用了没定义的变量就报错退出（防拼写错误悄悄放行）	脚本开头：全文安全网
`dirname / readlink -f`	dirname=取路径的目录部分；readlink -f=把相对路径展开成真实绝对路径	DIR= 行：无论从哪启动都能定位自己
`local`	声明“函数私有变量”，不污染函数外	parent_is_ours 里的 pid/spath 等
`case ... esac`	多分支判断（比一串 if 清晰）	args / STAT / 命令分发三处
`[[ ... =~ ... ]]`	正则匹配判断（=~ 后面的 ^[0-9]+$ 表示“纯数字”）	start_service 校验 PORT
`/proc/$pid/cwd`	/proc 是内核“现场直播”目录；cwd=该进程当前工作目录	三要素认领的第三要素
`kill -0`	不真发信号，只探“进程还活着吗”（活=返回0）	子服务退出检测、stop 轮询
`wait`	等待后台进程结束	stop_service 等子服务退出
`sed -n '1p'`	取文件的第 N 行（n=静默模式，1p=打印第1行）	读 PID 文件三行
`$$ / $! / $?`	$$=自己进程号；$!=最近后台进程号；$?=上一条命令退出码	写 PID 文件、SRV=$!、return 1
`trap`	收到信号时执行指定函数（TERM→stop_service）	start_service 里两行 trap
`2>/dev/null`	把错误输出丢进黑洞（安静失败）	大量命令后跟的尾巴

A2. labservice.conf（配置模板）

模板仅供查看格式，**不要复制占位值**。真实配置用这条命令生成（可照抄，$HOME 自动展开成真实路径）：`mkdir -p ~/lab/logs && echo "LOGDIR=$HOME/lab/logs" > labservice.conf && echo "PORT=8080" >> labservice.conf`。生成后用 **`cat labservice.conf`** 核对，应看到真实路径。

```
LOGDIR=<由上方生成命令产生，勿手写占位>
PORT=8080
```

A3. loop.sh（死循环进程，用于场景 4）

```
#!/bin/bash
# loop.sh - 练习用"卡死进程"（会吃满一个 CPU 核，用完记得 kill）
# 这行把 TERM 信号拦下（演示用），让 kill 对它无效—正是进4-2 要观察的现象
trap 'echo "忽略 TERM（演示用）"' TERM
while :; do :; done
```

loop.sh 跑起来会占满一个 CPU 核——用完必须 **`kill`** 掉，练习结束别忘清理。

附录 B • 本册命令速查

场景	命令一览
场景 1	`ps aux` · `grep` · `ss -tlnp` · `tail(-n/-f)` · `cat` · `nano`（vim 亦可） · `curl -I` · 处理流程四步

场景	命令一览
场景 2	`dd` · `mkfs.ext4` · `mount(-o loop)` · `umount` · `df -h` · `du -h --max-depth=1` · `ls -lhS` · `rm` · `fallocate`
场景 3	`ping(-c)` · `ip route` · `ss -tlnp` · `nc(-zv)` · `curl` · `kill` · 四步排查顺序
场景 4	`nohup` · `pgrep` · `top` · `kill/kill -9` · `fuser -v` · `nvidia-smi` · `systemctl(daemon-reload/enable/--now)` · unit 文件
场景 5	`#!/bin/bash` · 变量 · `if` · 注释 · `chmod +x` · `nc -z` · `stat -c %Y` · `date +%s` · `crontab(-e/-l)` · `>>` · `2>&1`

附录 C・术语表补充

术语	解释
沙盒式故障注入	在镜像文件/练习进程等"隔离环境"里复刻故障，不碰真实系统
挂载点（mount）	磁盘分区接入目录树的那个目录（如 /mnt/lab）
nohup	让进程忽略终端挂断信号：关了终端进程也不死
systemd / unit 文件	系统服务管家 / 描述"这个服务怎么启动"的配置文件
fuser	反查"是哪个进程占用着这个文件/设备"
nvidia-smi	NVIDIA 显卡状态查询工具（显存、占用进程）
cron / crontab	定时任务系统 / 编辑定时任务的命令（分 时 日 月 周）
处理流程四步	确定时间窗口 → 收集日志/状态/退出码 → 结合上下文定位 → 修复并复测

附录 D • 5 件套其余 3 个脚本（backup / disk_alert / service_guard）

场景 5 的"5 件套总览"里，check_res.sh 与 health_check.sh 在场景 5 题目中完成；本附录给出其余 3 个的完整实现与验证方式——每条命令的出处都标注了教科书章节，全部在教材体系内。（本附录给的是最小骨架、含退出码；2-12 验收要的「日志」在各脚本对应关卡里用 >> 追加补上即可。）

D1. backup.sh（自动备份 + 日期后缀）

```
#!/bin/bash
# backup.sh —— 把指定目录打包备份，文件名带日期+时间（不会覆盖旧备份）
# 用法：./backup.sh 要备份的目录（先 chmod +x backup.sh）
if [ "$1" = "" ]; then
  echo "用法：./backup.sh 要备份的目录"
  exit 1
fi
if [ ! -d "$1" ] || [ ! -r "$1" ]; then
  echo "源目录不存在或不可读：$1"
  exit 1
fi
STAMP=$(date +%F-%H%M%S)-$RANDOM
TARGET="$HOME/backup/backup-$STAMP.tar.gz"
mkdir -p "$HOME/backup"
if tar -czf "$TARGET" "$1"; then
  echo "备份完成：$TARGET"
  ls -lh "$TARGET"
else
  echo "备份失败：tar 返回非 0（检查源目录和磁盘空间）"
  exit 1
fi
```

正式跑起来终端会先出现一行 `tar: Removing leading `/' from member names`——那是 tar 把包内绝对路径改成相对路径的**正常提示**，不是报错，备份照样成功。

逐行要点：`[ "$1" = "" ]` 判断没传参数（= 在 [ ] 里是字符串比较，第 13 章）；`[ ! -d ]` 源目录不存在、`[ ! -r ]` 不可读，任一成立就退出——**源目录不存在绝不能打印"成功"**；`date +%F-%H%M%S` 输出"年-月-日-时分秒"（%F、%H、%M、%S 是固定格式符），**同一天重复跑也不会覆盖**；`tar -czf` 打包+压缩（c=建包 z=gzip f=文件名，第 12 章）；**if 直接判断 tar 命令**（命令返回 0=成功，非 0=失败，第 13 章），失败不打印成功、返回 1；`mkdir -p` 保证备份目录存在（第 2 章）；`exit 1` 异常退出（0 正常、非 0 异常，第 13 章）。**验证（五种情况）**：①正常备份 → 打印成功、文件存在；②不传参数 → 只打印用法、退出码 1；③源目录不存在 → 打印"不存在或不可读"、退出码 1；④权限不足 → 同上；⑤连续跑两次 → 两个不同名字的备份都在，不覆盖。退出码用 `echo $?` 查看。

`$RANDOM` = 每次给出 0~32767 的随机数（给文件名加唯一尾巴，避免重名覆盖）。

D2. disk_alert.sh（磁盘超 80% 告警）

```
#!/bin/bash
# disk_alert.sh —— 磁盘使用率超过 80% 就告警
# 用法：./disk_alert.sh（可放进 crontab 定时跑）
THRESHOLD=${THRESHOLD:-80}   # 有传就用传进来的，没传默认 80（例：THRESHOLD=90
./disk_alert.sh）
if ! df / > /dev/null 2>&1; then
  echo "df 执行失败（路径不存在或环境异常），退出"
  exit 1
fi
USE=$(df / | tail -1 | awk '{print $5}' | tr -d '%')
if [ "$USE" = "" ]; then
  echo "解析不到使用率（df 输出异常），退出"
  exit 1
fi
if ! [ "$USE" -eq "$USE" ] 2>/dev/null; then
  echo "使用率不是数字（$USE），退出"
  exit 1
fi
if [ "$USE" -gt "$THRESHOLD" ]; then
  echo "磁盘告警：根分区使用率 $USE%，超过 $THRESHOLD%"
else
  echo "磁盘正常：根分区使用率 $USE%"
fi
```

逐行要点：取数 → 判断 → 行动的巡检三步曲；`df /` 看根分区（第 7 章）、`tail -1` 取最后一行（第 3 章）、`awk '{print $5}'` 取第 5 列即使用率（第 4 章）、`tr -d '%'` 删掉百分号只剩数字（tr=字符转换、-d=删除，第 13 章 13.2）、`$(...)` 把命令输出装进变量（第 13 章）、`-gt` 在 [ ] 里表示"大于"（第 13 章）。**三层防护**：① df 本身失败先退出（路径不存在/环境异常）；② 解析结果为空先退出；③ `[ "$USE" -eq "$USE" ]` 是"数值自比较"技巧——USE 不是纯数字时 test 会失败（非 0），! 翻转后进分支退出，防止把垃圾当数字比大小。阈值放变量，以后想改只改一行。**验证**：正常时打印"磁盘正常：X%"；把 THRESHOLD 改成 0（或 THRESHOLD=0 ./disk_alert.sh 临时覆盖）再跑，应打印"磁盘告警"；把 df / 换成 df /nonexist 再跑，应打印"df 执行失败"且退出码 1——三种分支都出现即正确；建议配 crontab 每天跑一次（第 13 章）。

D3. service_guard.sh（服务挂了自动拉起）

```
#!/bin/bash
# service_guard.sh —— 服务挂了自动拉起（失败必须返回非 0，不伪装成功）
# 用法：./service_guard.sh 服务名（如 ssh、nginx）
if [ "$1" = "" ]; then
  echo "用法：./service_guard.sh 服务名"
  exit 1
fi
if systemctl is-active --quiet "$1" 2>/dev/null; then
  echo "$1 运行中，无需处理"
  exit 0
fi
echo "$1 没在运行，尝试拉起..."
if ! sudo systemctl start "$1" 2>/dev/null; then
  echo "$1 启动命令失败（服务不存在 / 配置错误 / systemd 不可用），看日志：journalctl -u $1"
  exit 1
fi
if systemctl is-active --quiet "$1"; then
  echo "$1 已拉起"
  exit 0
else
  echo "$1 拉起后仍不在运行（active/failed/offline 状态不同，逐一核实），看日志：journalctl -u
$1"
  exit 1
fi
```

逐行要点：保活三段——查（is-active 问"还活着吗"）→ 拉（死了就 start）→ 复核（拉完再问一次，没起来就看日志）。`systemctl is-active` 专门问服务状态，`--quiet` 安静模式只通过退出码回答（0=在，非 0=不在），正好放进 if（第 10 章）；`sudo systemctl start` 拉起需管理员权限（第 10 章）；`journalctl -u` 看该服务日志（第 10 章）。**返回码纪律**：成功（本来就在 / 拉起成功）返回 0；启动命令失败、拉起后仍 inactive/failed 都返回 1——**失败不伪装成功**，方便定时任务据此告警。`2>/dev/null` 把 systemd 不可用时的报错吞掉（第 4 章重定向），由退出码分支统一处理。**验证**：示例服务要挑**你机器上确实存在**的（如 `systemd-journald` 或 `cron`；**没装 ssh 服务的话别用它**）——`./service_guard.sh systemd-journald` 输出"运行中"、退出码 0；先 `sudo systemctl stop systemd-journald`（或换一个你能停的服务）再跑，应看到"没在运行 → 尝试拉起 → 已拉起"、退出码 0；对一个不存在的服务名跑，应打印"启动命令失败"、退出码 1（ 需电脑 Ubuntu；systemd 不可用的容器/WSL 记为环境限制）。

（2-09「参数与用法：$1 / usage」的示范就在本册：`[ "$1" = "" ]` 的用法分支 + `case "$args"` 的写法——做 2-09 时以这里的脚本为准。要「--help」效果：把"缺参数给用法"那段加一行 `[ "$1" = "--help" ]`（或并进 case）打印同样的用法文字就行。另：脚本一旦改名，用法文字、日志文件名和 crontab 里的旧名字都要同步改——不然 --help 会打出已经不存在的旧名。）

阶段总结练习（新增 • 打完本阶段关卡再做）

阶段 2 • 学习总结练习：5 件套脚本 + crontab + systemd（自愈的雏形）

业务场景：组里没有监控系统，同事说"至少磁盘快满时有人知道、服务挂了能自己起来"。

综合要求：① backup.sh 带日期后缀、路径含空格也能跑；② disk_alert.sh 超阈值输出告警；③ service_guard.sh 检测到服务挂掉就拉起；④ 一个脚本进 crontab（绝对路径）、一个进 systemd；⑤ 三个脚本都有日志与退出码；⑥ 仓库带 README（用法 + 最小复现步骤）。

验收：□ 陌生人照 README 能跑通三个脚本　□ 定时任务真的执行过（日志为证）　□ 手动停服务后能被拉起　□ 每个脚本能讲 3 分钟

时间：4 小时（休息日）　**产出**：shell-scripts 仓库（本练习做 3 个新脚本：backup / disk_alert / service_guard + README；另 2 个 check_res / health_check 在场景 5 已完成——合起来就是 5 件套）

AI 运维指向：这是监控与自愈的最小版本——以后 GPU 机器上的"显存超阈值告警""推理服务挂了自动拉起"用的是同一套骨架。

实战价值：实战里最看重的"讲一件真正动手做过的事"，这三个脚本最容易讲清楚。